DEPARTMENT 1 · DEVELOPMENT AND IT · PROFESSION 1

Full-stack developer

Profession 1 of 10 in Development and IT.

What the profession means

A full-stack developer builds both sides of a web application: the screens people see (front end) and the server, database and logic behind them (back end).

What professionals do every day

Plans features, builds screens, writes API endpoints, designs tables, fixes bugs, writes tests and deploys the application. Works with git, reads error logs and talks to clients about what they need.

Why businesses hire them, and what problems they solve

A business with a booking, shop or dashboard needs one person who can carry a feature from the database to the button. Full-stack developers reduce hand-offs and can deliver a complete product.

- Turning a client's idea into screens, data and rules.
- Keeping data safe and consistent between the screen and the database.
- Shipping a working version quickly, then improving it.

Skills you need

- HTML, CSS and JavaScript (Part 1, Module B).
- A front-end framework such as React (see Profession 9).
- Node.js and an API framework.
- Databases (SQL and at least one hosted option).
- Authentication, validation and error handling.
- Git, testing and deployment.

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
Visual Studio Code	Code editor	Free	Free	code.visualstudio.com
Node.js	Runs JavaScript on the server	Free	Free	nodejs.org
GitHub	Code hosting and collaboration	Free tier	Paid team plans	github.com
Supabase	Hosted Postgres database and authentication	Free tier	Paid plans	supabase.com
Vercel	Hosting for front-end and serverless back-end	Hobby tier	Paid plans	vercel.com

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	Weeks 1–4: HTML, CSS, JavaScript, git. Build the bakery site from Part 1.
Intermediate	Weeks 5–10: React basics, Node API, one database table, form to database.
Advanced	Weeks 11–20: authentication, tests, deployment, a client-style feature with a real data flow.

Projects by level

- **Beginner:** A notes app with add, list and delete, stored in a file or database.
- **Intermediate:** A booking form that saves to a database and sends an email confirmation.
- **Advanced:** A multi-user dashboard with sign-in, roles and an admin view.

Client-ready portfolio project

A complete application for a small business: sign-in, a list of records, a form, a database, deployment on a public address and a README that explains how to run it. Use a real business (with permission) or a clearly labelled fictional one.

Worked example

Worked example. A clinic wants a 'book a visit' page. The chain is: form (front end) sends a POST to `/api/bookings` (back end), the server validates the date and phone number, the database stores the row, and the page shows a confirmation. Each part can be tested alone.

Assignment with full solution

ASSIGNMENT

A notes API with validation

Business problem: A small team needs a notes service for its website. The API must list notes, add a note, and reject empty text.

Steps

1. Create `server.js` as shown in the solution. Start it with `node server.js`.
2. Test with a browser at `/api/notes` (GET).
3. Send a POST with `{"text": "Buy milk"}` and check for status 201.
4. Send a POST with empty text and check for status 400.

Expected output: A GET returns a JSON list; a valid POST returns 201 with the new note; an empty text returns 400.

SOLUTION (ANSWER KEY)

The solution uses only Node's built-in `http` module, so nothing needs installing. Each rule maps to a status code: 200 for reading, 201 for creating, 400 for bad input, 404 for unknown routes. Tested with `curl -X POST -d '{"text":""}'`, which returns 400.

JAVASCRIPT

```
// server.js  -- a tiny JSON API using only Node's built-in http module
const http = require('http');
let notes = [{ id: 1, text: 'Call supplier' }];
let nextId = 2;
const send = (res, status, body) => {
  res.writeHead(status, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(body));
};
http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/api/notes') return send(res, 200, notes);
  if (req.method === 'POST' && req.url === '/api/notes') {
    let raw = '';
    req.on('data', (c) => (raw += c));
    req.on('end', () => {
      let data;
      try { data = JSON.parse(raw); } catch { return send(res, 400, { error: 'invalid JSON'
}); }
      if (!data.text || !data.text.trim()) return send(res, 400, { error: 'text is required'
});
      const note = { id: nextId++, text: data.text.trim() };
      notes.push(note);
      send(res, 201, note);
    });
    return;
  }
  send(res, 404, { error: 'not found' });
}).listen(3000, () => console.log('API on http://localhost:3000'));
```

Common mistakes

- Writing the database and API before agreeing the screens with the client.
- Storing passwords in plain text. Always hash them and use an authentication service.
- Skipping validation on the server, which trusts only the browser.
- Deploying with secret keys in the code.

Freelancing opportunities

- Freelance web application builds for small businesses: booking systems, portals, dashboards.
- Maintenance and feature retainers after launch.
- Starting points on freelance marketplaces (see Freelancing chapter): search for 'full stack', 'React', 'Node', 'API' jobs and apply with a working demo.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- MDN: Learn web development
- Node.js learning resources
- The Odin Project (free full-stack curriculum)
- Supabase documentation

Part of **Master Freelancing: Development and IT**. Back to the chapter list on skillnavora.in · Open the full textbook. Official links in this chapter were not checked live; confirm before relying on them.